

MiniWear: Minimizing Flash Wear via Hybrid Persistent Cache for Extended EF-SMR Lifetime

Chenlin Ma¹, Kaoyi Sun¹, Yuxuan Qi¹, Jiaxian Chen¹, Xiaochuan Zheng¹, Tianyu Wang¹ and Yi Wang^{1,2}

¹ College of Computer Science and Software Engineering, Shenzhen University

² State Key Laboratory of Radio Frequency Heterogeneous Integration

Abstract—As the huge discrepancy between traffic and capacity persists, the lifetime of flash in EF-SMR systems faces a grave issue. EF-SMR systems combine NAND flash with Shingled Magnetic Recording (SMR) disks to achieve both low cost and high performance. However, previous research has primarily focused on issues such as *write amplification* and *tail-latency* in EF-SMR disks, overlooking the critical issue of flash lifetime. Studying the durability of EF-SMR systems is essential for developing future high-performance, low-cost storage solutions.

This paper presents *MiniWear*, a hybrid persistent cache (PC) design aimed at extending the lifetime of EF-SMR systems. MiniWear adopts a hybrid medium persistent cache and proposes a customized scheduling strategy to reduce flash wear without impacting the EF-SMR system performance. At the hardware level, the hybrid PC of EF-SMR, composed of flash and SMR disk, is organized into Flash-PC and SMR-PC. At the software level, a fine-grained scheduling strategy is proposed to better manage PC resources. Additionally, we introduce a proactive balancing strategy to address PC resource idleness. Experimental results show that, compared to existing methods, MiniWear can reduce flash wear by up to 66.67%.

Index Terms—Lifetime, Embedded Flash, Shingled Magnetic Recording, Cache Management.

I. INTRODUCTION

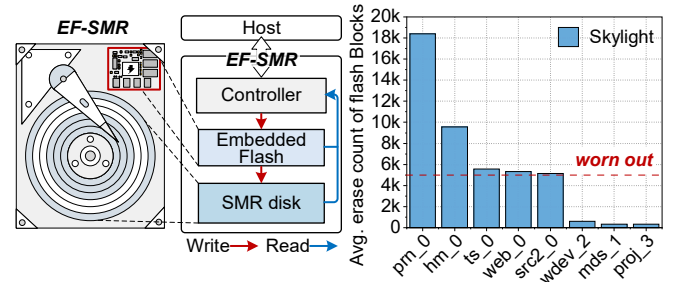
Shingled Magnetic Recording (SMR) is a promising technique to promote disk density to more than 10 Tbits/in² by overlapping tracks one on another like shingling on a roof [1], [2]. As a side effect of the high density, SMR disks have inherent *write amplification* and *tail-latency* issues caused by Read-Modify-Write (RMW) operations. RMW operations can accommodate random writes but also drive the read/write head to move back and forth frequently, which significantly degrade the system's performance [3], [4]. Embedded Flash with SMR disk (EF-SMR) [5] integrates high-performance NAND flash memory, where random write data is first buffered in the flash before being written to the disk through RMW operations. However, the flash memory in EF-SMR systems faces significant write pressure, as shown in Fig. 1(a), and once the flash is occupied, it needs to be erased by a blocking background garbage collection (GC) process. The frequent GC processes lead to repeated wear of flash blocks, thereby introducing the system's *lifetime* issue.

We have conducted a preliminary experiment to demonstrate flash memory wear in the EF-SMR system. As Fig. 1(b) shows, the average erase count commonly exceeded 5k, reaching the upper limits for Multi-Level Cell (MLC) flash memory [6]. Considering the limited durability of NAND flash in erase cycles and the terabyte-scale write traffic in EF-SMR systems,

our findings indicate that the *lifetime* of EF-SMR disks faces significant challenges.

Most previous works aim to resolve *write amplification* and *tail-latency* issues [7]–[15]. For example, PORE [12] implements a Partial Open-zone Eviction Strategy to mitigate SMR's write amplification by better balancing cache access frequencies. ROCO [13] uses a solid state drive (SSD) cache that absorbs random writes to reduce the write amplification of the Host-Aware SMR disk. HS-BAS [16] also utilizes SSD caching to diminish the collection costs associated with SMR's internal persistent cache. MAID-Q [14] targets to minimize tail latency by adopting a lightweight reinforcement learning model. MCM [15] reduces flash erasures by partitioning data into cold and hot segments. Unlike previous works, this paper proposes a *hybrid persistent cache* architecture designed to extend the *lifetime* of the EF-SMR system.

This paper introduces *MiniWear*, a design for the EF-SMR system. Our main objective is to minimize flash wear via hybrid persistent cache, thereby extending the overall lifetime of the EF-SMR system. MiniWear incorporates hardware design and software optimization. The hardware design adopts a new persistent cache architecture with a hybrid medium, defining specific track zones of the SMR disk as SMR-PC. It also establishes a hybrid persistent cache mapping to distribute write stress to SMR-PC, effectively reducing the writing pressure faced by the flash. Based on the hardware design, the software optimization performs a fine-grained scheduling strategy of the persistent cache (PC), achieving efficient allocation of persistent cache resources. This ensures MiniWear can extend the lifetime of the EF-SMR system without degrading performance. In terms of system lifetime, MiniWear is compared with representative benchmark methods, showing a maximum lifetime extension of 66.67%.



(a) EF-SMR system.

(b) Average erase count.

Fig. 1: Illustration of EF-SMR system.

The major contributions include:

- *Hybrid Persistent Cache Architecture*: A hybrid persistent cache designed for the EF-SMR system, along with a customized mapping that reduces the write traffic flowing through the flash memory.
- *Scheduling Strategy*: A scheduling strategy is proposed, which enables a fine-grained allocation of PC resources to extend the lifetime of the EF-SMR system further.
- *Experiments*: Comprehensive method validation through a comparative analysis with representative solutions.

The rest of this paper is organized as follows: Section II provides background and discusses the motivation of this paper. Section III presents our proposed MiniWear in detail. Section IV presents experimental results. Finally, Section V concludes this paper and discusses future work.

II. BACKGROUND AND MOTIVATION

A. Shingled Magnetic Recording Disk

In the era of big data, there is an exponential growth in data generation [17], [18]. The emergence of the Internet of Things (IoT), Artificial Intelligence (AI), and other data-intensive technologies has accelerated this trend, leading to a continuous increase in the demand for storage space. Traditional storage solutions, such as Conventional Magnetic Recording (CMR) disks, as shown in Fig. 2(a), struggle to meet the increasing demands due to their limited storage density. This challenge has led to a need for more advanced storage solutions. Shingled Magnetic Recording (SMR) drives are one such solution, offering higher storage density by overlaying magnetic tracks on top of each other in a “shingled” manner, as illustrated in Fig. 2(b). In essence, while CMR maintains gaps between tracks to avoid interference, SMR maximizes space utilization by overlapping magnetic tracks to address the extensive storage demands [19].

B. Embedded Flash with SMR Disk

SMR overlaps track on previous tracks, like shingling the roofs to break through the areal density limitation of conventional hard disk drives (HDDs) [20]–[24]. However, this shingled-like structure incurs an accidental data loss on the tracks due to the write head is wider than the track width. Writing to a specific track can destroy the valid data stored on the adjacent tracks. To avoid accidental data loss, SMR requires an approach named Read-Modify-Write (RMW) operation to address random writes. The RMW operation will

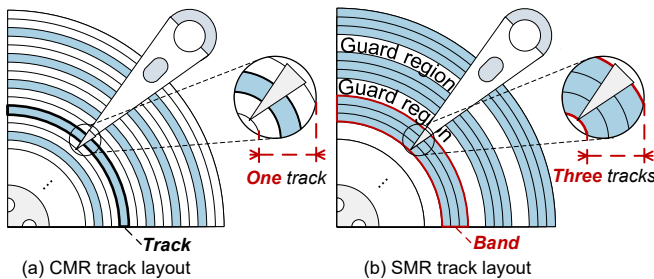


Fig. 2: SMR overlaps magnetic tracks within a Band to achieve higher storage density compared to CMR.

first read all valid data on the Band, and after modifying with the randomly written data, the modified data will be written back to the Band sequentially.

Since every RMW operation needs to overwrite the large-size SMR Band, it can lead to the write amplification issue and degrade the system performance. To achieve satisfactory run-time performance, several prior works [5], [12], [13], [25] integrate a NAND flash as a first-level persistent cache (PC) to form a faster EF-SMR disk. All random written data will be provisionally cached into the flash for high performance. When the flash is full, the valid data cached in the flash will be migrated to corresponding SMR Bands through a decent number of RMW operations. However, the capacity of the embedded flash is only about 1% of the total capacity of the EF-SMR disk [5]. Therefore, this indiscriminate data storing into flash puts severe wear pressure on the embedded flash, making it quickly worn out and unusable [26].

C. Motivation

To maintain low costs, the capacity of embedded flash memory in EF-SMR system is limited to 1% of the total capacity [5]. The existing EF-SMR system caches all random write requests to the flash, leading to durability issues, as shown in Fig. 3(a). Specifically, the limited flash capacity handles a large volume of write traffic, causing frequent programming and erasure of flash blocks to continuously satisfy write requests, thereby resulting in significant wear pressure. On the other hand, as each cell can store more bits by the multi-level cell technology, the durability of flash memory cells deteriorates [27]–[30]. For instance, the maximum P/E cycles of single-level cell (SLC) are about 10k, while those of triple-level cell (TLC) are decreased to 3k [6].

The limited flash capacity is the fundamental reason affecting the lifetime of the EF-SMR system, but increasing the flash capacity would raise costs. Therefore, there is an urgent need for a hybrid medium persistent cache in the EF-SMR system, as illustrated in Fig. 3(b). EF-SMR blurs the boundaries between flash and SMR disk by offering low latency and large capacity. Thus, the challenge of adding a hybrid medium persistent cache lies in how to extend the lifetime of the EF-SMR system as much as possible without sacrificing read/write performance. To address the challenge, this paper proposes MiniWear, which achieves this goal by designing a hybrid persistent cache and resource scheduling strategy.

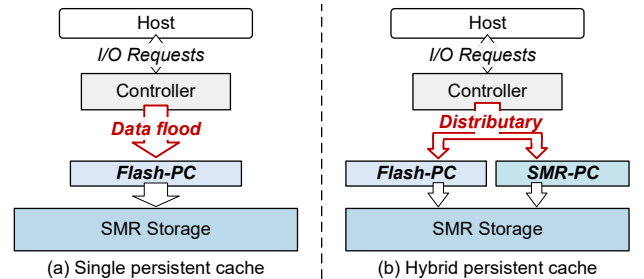


Fig. 3: Single PC utilizes only flash medium, while a hybrid PC incorporates both flash and SMR disk medium.

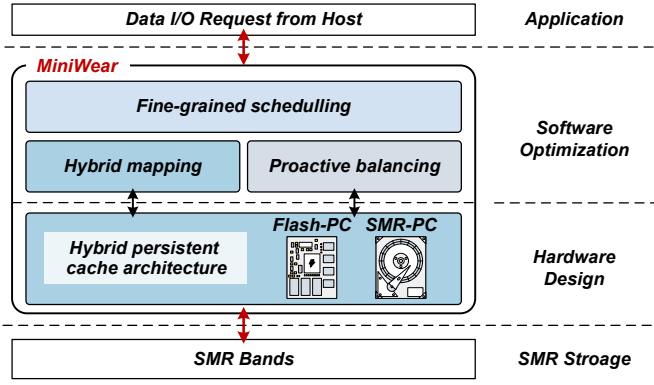


Fig. 4: The system overview of MiniWear.

III. MINIWEAR: A HYBRID PERSISTENT CACHE DESIGN FOR EMBEDDED FLASH WITH SMR DISK

A. Overview of MiniWear

MiniWear adopts a hardware/software co-design, operating between applications and SMR storage devices to provide data scheduling, as shown in Fig. 4. The system comprises three main components: hybrid persistent cache architecture, fine-grained scheduling, and proactive balancing. The hybrid persistent cache provides hardware support. This foundation allows for a fine-grained scheduling strategy to ensure rational resource allocation. If PC resources are improperly allocated, a proactive balancing strategy adjusts and enhances the system. The hybrid address mapping and scheduling strategies are implemented through the PC manager. These three components are interconnected, forming the core of MiniWear. The hybrid persistent cache consists of Flash-PC and SMR-PC. Introducing SMR-PC helps reduce writing pressure on flash. The fine-grained scheduling strategy manages persistent cache allocation and reclamation, optimizing resource use and minimizing flash wear. The proactive balancing strategy recycles underutilized Flash-PC resources, improving utilization and preventing the cycle of frequent garbage collection due to shortages. The detailed designs of these components are as follows.

B. Hybrid Persistent Cache Architecture

In MiniWear, the hybrid persistent cache architecture consists of two parts: high-performance NAND flash (Flash-PC) and a few designated outer tracks on SMR disk platters (SMR-PC). In the following, we describe Flash-PC, SMR-PC, and the mapping strategy.

Flash-PC: Flash-PC consists of NAND flash memory, with the capacity of 1% of the EF-SMR system, which uses “Block” as the basic unit for scheduling, as shown in Fig. 5(a). MiniWear also adopts this approach, defining a flash block as the unit for Flash-PC scheduling, where one flash block represents one Flash-PC resource. However, the scheduling strategy for flash blocks in MiniWear differs from that of other works.

SMR-PC: We have innovatively integrated SMR-PC into EF-SMR, using a hardware-software co-design approach to ex-

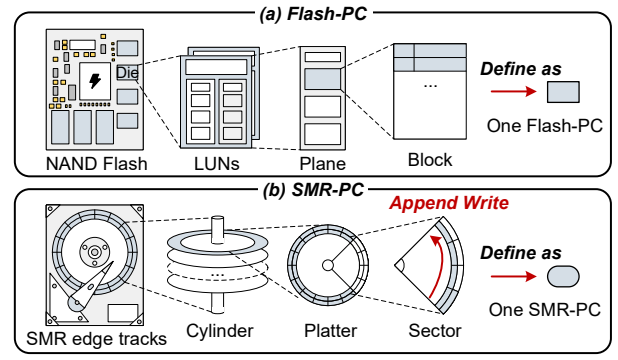


Fig. 5: Hardware design of hybrid persistent cache.

tend the system’s lifetime without compromising performance. SMR-PC consists of 1% of the outer tracks of the SMR disk, which supports random writes and offers better performance compared to the SMR Bands. As shown in Fig. 5(b). In MiniWear, SMR-PC is divided into equal-sized sectors, with each sector serving as a unit for SMR-PC scheduling. We designed a principle for SMR-PC that caches data using an append-write approach. The mapping relationships are recorded in a mapping table. For repeatedly written data, the append-write approach is used, and the mapping relationships are updated, with the original data automatically becoming invalid. This provides a high-performance caching mechanism to deal with random writes on the SMR disk. This design eliminates the write amplification, ensuring the superior performance of the EF-SMR system.

Hybrid Mapping: To effectively utilize the hybrid persistent cache, MiniWear introduces a hybrid mapping from the logic address to the address of PC or SMR. A page-level mapping table is used to record the mapping relationship. A logical page address may be mapped to SMR, Flash-PC or SMR-PC. MiniWear records whether the logical page address is mapped to the PC through the entry in the mapping table. When mapped to SMR, the physical address is the same as the logical address. Otherwise, the corresponding PC type and physical page address are recorded in the entry. Besides a page-level mapping table, there are two priority queues and bifurcated linked lists to handle mapping relationships efficiently. Priority queues record the starting addresses of available resources in Flash-PC and SMR-PC for quick lookup. A bifurcated linked

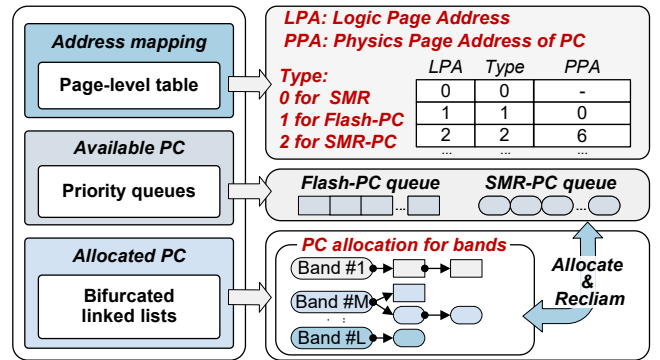


Fig. 6: Hybrid mapping.

list for each Band records the ownership status of both Flash-PC and SMR-PC resources. With the support of these data structures, MiniWear can efficiently traverse available PC and PC used by a specific Band when scheduling. The memory overhead caused by the mapping tables, priority queues, and linked lists will be analyzed in subsection III-E (Memory Overhead).

C. Fine-Grained Scheduling

The fine-grained scheduling strategy is designed for the hybrid persistent cache we defined. This strategy is executed by the PC controller, operating within the translation layer of the EF-SMR system. It controls the *allocation* and *reclamation* of PC to minimize wear on Flash-PC as much as possible, as shown in Algorithm 1.

Allocation: PC is allocated exclusively at the Band level, meaning that once PC is allocated to a Band, it is dedicated to that Band until reclaimed. To ensure the optimal performance of the EF-SMR system, the PC manager prioritizes the allocation of Flash-PC resources, as shown in Fig. 7(a). When Flash-PC resources are insufficient, SMR-PC resources are prioritized, as illustrated in Fig. 7(b). Resource allocation failure can occur in two scenarios: (1) when both Flash-PC and SMR-PC resources are exhausted, and (2) when the SMR Band reaches the maximum limit of SMR-PC resources. Since SMR-PC is less efficient than Flash-PC, its usage should be appropriately restricted. When allocation fails, the system performs garbage collection and reallocates resources after the collection process is complete.

Algorithm 1: Fine-Grained Scheduling

Input: LSN: Logical Sector Number and data of write request
Output: write result

```

1: // Allocation Strategy
2: LBN ← LSN / sector_of_Band
3: fpc_w ← Flash-PC_Writable[LBN]
4: spc_w ← SMR-PC_Writable[LBN]
5: if not fpc_w then
6:   fpc_w ← Distribute_Fpc()
7: end if
8: if not fpc_w and not spc_w then
9:   spc_w ← Distribute_Spc()
10: end if
11: // Reclamation Strategy
12: if not fpc_w and not spc_w then
13:   fpc_w ← Erase_and_Allocate(Find_Obsolete())
14:   if not fpc_w then
15:     fpc_w, spc_w ← Partial_Reclaim(LBN, longest_Band)
16:   end if
17:   if not fpc_w and not spc_w then
18:     Full_Reclaim(longest_Band)
19:   end if
20: end if
21: // Write
22: if fpc_w then
23:   Flash-PC_Write(DATA)
24: else
25:   SMR-PC_Write(DATA)
26: end if

```

Reclamation: Garbage collection tasks are also controlled by the PC manager, and we categorize garbage collection into three levels: obsolete flash block reclamation, Band-level partial reclamation, and Band-level full reclamation.

Obsolete flash block reclamation: Obsolete blocks refer to allocated flash blocks whose data has become entirely invalidated. Reclaiming obsolete flash blocks can rapidly complete the garbage collection process because it does not require data migration from persistent cache to SMR storage. This maximizes the performance of the EF-SMR system. During this operation, the manager only needs to locate an obsolete block and erase it to complete the current GC process, as shown in Fig. 7(c). **Band-level partial reclamation:** This operation reclaims all SMR-PCs owned by the Band and a portion of flash blocks (whose invalidation rate exceeds a specific threshold, which can be adjusted as needed; for example, set at 25%). The manager sequentially checks whether the Band currently being written to and the Band with the most flash resources meet the criteria for partial reclamation, and reclaims resources from the first Band to meet these criteria, as shown in Fig. 7(e) and (f). **Band-level full reclamation:** If none of the above conditions are met, the SMR Band with the most Flash-PC resources is chosen as the victim, and all its associated PC resources are completely reclaimed, as shown in Fig. 7(d).

Fine granularity is reflected in the individual and partial reclamation of flash blocks. Compared to complete reclamation, these two methods allow for more detailed consideration of the system's state, facilitating more nuanced GC processes. Obsolete blocks contain no valid data, and their reclamation does not involve data write-back operations, maintaining optimal system performance. Reclaiming a single invalid block significantly reduces the number of flash erasures compared to complete reclamation; partial reclamation aims to minimize the number of flash blocks erased during each GC process as much as possible.

D. Proactive Balancing

This strategy alleviates the issue of resource scarcity by proactively reclaiming underutilized flash resources, executed by the PC manager. In the EF-SMR system, there are underutilized flash blocks, also known as “zombie blocks”, which are allocated on-demand to SMR Bands but are subsequently seldom used. This directly reduces the total amount of available Flash-PC, forcing more frequent garbage collection processes and indirectly impairing the system's lifetime.

$$T_i = \frac{Access_i}{NumFpc_i} \quad (1)$$

The strategy is implemented as follows: it is executed when the number of passive GC processes reaches a preset threshold (an adjustable empirical value). To identify the victim Band for proactive reclamation, we propose a metric T to estimate the hotness of Bands. This metric is determined by the relationship between the number of accesses to the Band during the interval between two proactive reclamations and the number of Flash-PC resources owned by the Band. Defined in Equation 1, T_i

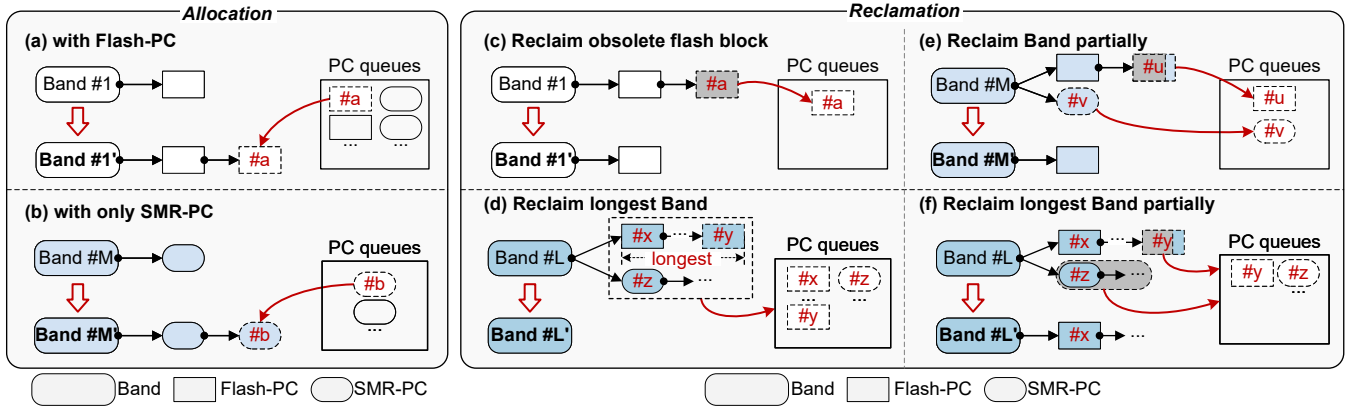


Fig. 7: Fine-grained scheduling.

represents the hotness of $Band_i$, $Access_i$ denotes the number of data accesses for $Band_i$, during the interval between two proactive reclamations, $NumFpc_i$ is the total number of Flash-PC resources owned by $Band_i$ (excluding Bands that do not have Flash-PC). Ultimately, the manager is responsible for reclaiming all PC from the Band with the lowest hotness.

E. Overhead Analysis

Memory Overhead: Flash-PC and SMR-PC each account for 1% of the total SMR Band capacity, resulting in a total persistent cache capacity of 40.96 GiB for a 2 TB SMR device. Specifically, the page-level mapping table requires 40.96 MiB (assuming a 4 KiB page size and 4 B page address), while the priority queue and linked list for Flash-PC consume 640 KiB and 1.28 MiB, respectively (with each linked list node occupying 8 B). The overhead for SMR-PC is similar to that of Flash-PC, and the total memory consumption remains acceptable for EF-SMR systems.

Computational Overhead: For fine-grained scheduling and proactive balancing strategies, we propose several algorithms. The time complexity of these algorithms is $\mathcal{O}(n)$, which does not impose an excessive burden on computational resources, making it acceptable for the system.

IV. EVALUATION

A. Experimental Setup

Baseline. We compare MiniWear with Skylight [5], PORE [12], MCM [15] and Duchy [25]. Skylight uses static address mapping to integrate flash memory into SMR disks. PORE reduces I/O latency by limiting the reclamation scope and aggressively using flash resources. MCM differentiates between sequential hot/cold writes and random writes to improve latency and lifetime. Duchy proposes a write filtering scheme to enhance SSD endurance.

Performance Metrics. we compare MiniWear to Skylight [5], PORE [12], MCM [15] and Duchy [25] with the following performance metrics: (1) flash-based PC wear; (2) response time; (3) write amplification;

Parameter Setting. The specific configurations of the EF-SMR are set according to the observation from the realistic

SMR disk [5] and Samsung NAND flash chip [31], which are depicted as follows: (1) The capacity of the persistent cache is 2% of the SMR disk capacity, with Flash-PC and SMR-PC each accounting for 1%. (2) The rotation speed of the disk is set as 7200 rpm, and the data transfer time is 0.03 ms; (3) The width of the write head is three times that of the read head (i.e., $K=3$); (4) The Band/Zone size is set as 40 MiB, totaling 256 in number; (5) The flash operation latencies for one page read, one page write and one block erase are 0.2 ms, 1.3 ms, and 1.5 ms, respectively. (6) The outermost circular tracks used as SMR-PC resources are logically divided into 128 sectors of equal capacity;

Datasets. We employed eight publicly accessible datasets characterized by varying sizes and access patterns, as elucidated in Table I. These datasets have been widely utilized for benchmarking the EF-SMR, as referenced in prior works [32].

B. Result and Discussion

Flash-based PC wear: As shown in Fig. 8(a), we evaluated the average wear count of the flash memory across different datasets and normalized the results. It can be observed that, on average, MiniWear reduces the flash-based PC wear count by 20.08%, 34.82%, 33.76%, and 32.93%, with maximum reductions of 30.67% (i.e., proj_4), 66.67% (i.e., web_2), 58.79% (i.e., web_2), and 62.52% (i.e., web_2), compared to Duchy, MCM, PORE, and Skylight, respectively. The reason for the optimization of flash memory lifetime is the introduction of the new medium (SMR-PC), which alleviates the write pressure on the flash and reduces its erasures. To further investigate the performance under more limited flash, we evaluated the average wear count of the flash memory with different flash capacities and normalized the results. As shown in Fig. 8(b), it can be observed that as the flash capacity decreases, the wear count increases for all methods. However, MiniWear shows a slower rate of increase, indicating that it has the capability to cope with extreme flash wear.

Response time: The average response time is a key indicator of system performance. We collected average response time data for quantitative analysis and normalized the data. As shown in Fig. 9(a), it can be observed that MiniWear reduces

TABLE I: The characteristics of the traces.

Traces	Data Written (GB)	Data Read (GB)	Write Ratio	Update Ratio
hm_0	9.83	5.41	64.50%	97.72%
mds_1	0.45	5.80	7.12%	79.99%
prn_0	19.01	2.30	89.21%	94.52%
proj_3	0.44	8.12	5.18%	65.61%
proj_4	0.37	24.30	1.48%	88.17%
web_0	5.43	2.31	70.12%	99.21%
web_1	0.28	0.33	45.89%	91.69%
web_2	0.15	19.59	0.75%	52.49%

the average response time by 4.14%, 30.21%, 32.33%, and 53.21%, respectively, compared to Duchy, MCM, PORE, and Skylight. The maximum reduction rates reach 27.59% (hm_0), 64.08% (web_0), 67.55% (proj_3), and 83.48% (proj_3). This improvement is attributed to the fine-grained scheduling strategy, which leverages the architectural advantages of the hybrid persistent cache. Additionally, we analyzed the frequency distribution of response times for each request, as shown in Fig. 9(b), where the x-axis represents increasing response times. MiniWear exhibits the lowest frequency of high response times, indicating greater stability. This stability results from the ability of MiniWear to minimize unnecessary RMW operations during garbage collection.

Observant readers may notice that in Fig. 9(a), for read-intensive workloads (e.g., mds_1), the average response time of MiniWear is slightly higher than that of Duchy. This phenomenon is attributed to the proactive balancing strategy, which involves reclaiming additional flash blocks. This approach can cause fluctuations in I/O latency for read-intensive workloads.

Write amplification: Write amplification is defined as the ratio of the actual amount of data written to the original input data volume. The optimization goal is to minimize this ratio to ensure system performance. We collected write amplification data from Duchy, MCM, PORE, and Skylight and normalized it to our proposed MiniWear. As shown in Fig. 10(a), MiniWear significantly reduces write amplification across all write-intensive and medium write volume traces compared to the baseline schemes. When handling read-intensive traces (e.g., proj_3), MiniWear exhibits slightly higher write amplification than Duchy. This is due to the proactive reclamation process involving RMW operations, which tend to increase write amplification. Additionally, we analyzed the actual data written for the same set of requests, with results summarized in

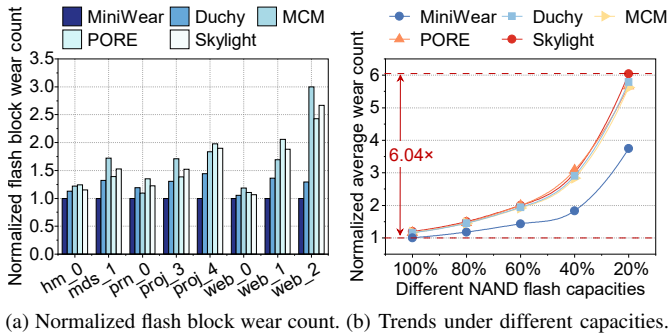


Fig. 8: Flash-based PC wear.

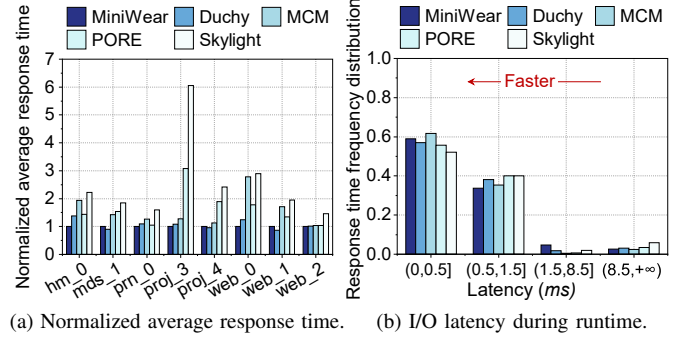


Fig. 9: Response time.

Table II. The data indicate that MiniWear effectively reduces the actual amount of data written, highlighting its capability to minimize unnecessary writes.

V. CONCLUSION

In this paper, we explore the issue of minimizing wear on NAND flash within EF-SMR disk storage systems. We introduce MiniWear, a novel hybrid persistent cache intermediate layer for the EF-SMR system. By introducing SMR-PC and optimizing address mapping and resource scheduling strategies, MiniWear can effectively reduce the number of flash erasures and extend the system's lifetime. Experimental results demonstrate its superior performance. In the future, we plan to optimize the Wear-leveling module to address "zombie blocks" with finer granularity. We also plan to investigate alternative approaches to partitioning SMR-PC so that MiniWear can perform better under various workloads.

ACKNOWLEDGEMENTS

The work described in this paper is supported in part by NSFC (U23B2040), in part by National Key Research and Development Program of China (2024YFB4505702), in part by Shenzhen Science and Technology Program (RCJC20221008092725019), in part by Guangdong Province Key Laboratory of Popular High Performance Computers (2017B030314073), in part by Guangdong Province Engineering Center of China-made High Performance Data Computing System and Shenzhen Key Laboratory of Service Computing and Applications. Tianyu Wang is the corresponding author.

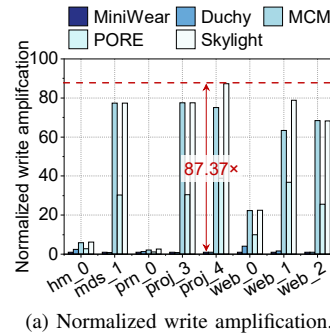


TABLE II: Actual written.

Method	Volume (GB)
MiniWear	914
Duchy	1,370
MCM	3,128
PORE	1,687
Skylight	3,595

Fig. 10: Write amplification.

REFERENCES

- [1] D. Sun and Y. Chai, "SAC: A Co-Design Cache Algorithm for Emerging SMR-Based High-Density Disks," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 1047–1061.
- [2] C.-F. Wu, M.-C. Yang, and Y.-H. Chang, "Improving Runtime Performance of Deduplication System with Host-Managed SMR Storage Drives," in *Design Automation Conference (DAC)*, 2018, pp. 1–6.
- [3] C. Jin, W.-Y. Xi, Z.-Y. Ching, F. Huo, and C.-T. Lim, "HiSMRfs: A High Performance File System for Shingled Storage Array," in *Mass Storage Systems and Technologies (MSST)*, 2014, pp. 1–6.
- [4] C. Ma, Z. Shen, L. Han, R. Chen, and Z. Shao, "FC: Built-in Flash Cache with Fast Cleaning for SMR Storage Systems," *Elsevier Journal of Systems Architecture (JSA)*, vol. 98, pp. 214–220, 2019.
- [5] A. Aghayev, M. Shafaei, and P. Desnoyers, "Skylight—A Window on Shingled Disk Operation," in *Conference on File and Storage Technologies (FAST)*, 2015, pp. 1–16.
- [6] J. Wu, J. Li, Z. Sha, Z. Cai, and J. Liao, "Adaptive Switch on Wear Leveling for Enhancing I/O Latency and Lifetime of High-Density SSDs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 11, pp. 4040–4051, 2022.
- [7] C. Ma, Z. Zhou, Y. Wang, Y. Wang, and R. Mao, "MU-RMW: Minimizing Unnecessary RMW Operations in the Embedded Flash with SMR Disk," in *Design, Automation, and Test in Europe (DATE)*, 2022, pp. 490–495.
- [8] X. Xie, L. Xiao, X. Ge, and Q. Li, "SMRC: An Endurable SSD Cache for Host-Aware Shingled Magnetic Recording Drives," *IEEE Access*, vol. 6, pp. 20916–20928, 2018.
- [9] B. Zhang, M.-H. Yang, X. Xie, and D. H. Du, "Idler: I/O Workload Controlling for Better Responsiveness on Host-Aware Shingled Magnetic Recording Drives," *IEEE Transactions on Computers (TC)*, vol. 69, no. 6, pp. 777–788, 2020.
- [10] C. Ma, Y. Wang, Z. Shen, and Z. Shao, "KFR: Optimal Cache Management with K-Framed Reclamation for Drive-Managed SMR Disks," in *Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [11] S. Huang, Q. Wei, D. Feng, J. Chen, and C. Chen, "Improving Flash-Based Disk Cache with Lazy Adaptive Replacement," *ACM Transactions on Storage (TOS)*, vol. 12, no. 2, pp. 1–24, 2016.
- [12] C. Wang, D. Wang, Y. Chai, C. Wang, and D. Sun, "Larger Cheaper But Faster: SSD-SMR Hybrid Storage Boosted by a New SMR-Oriented Cache Framework," in *Conference on Mass Storage Systems and Technologies (MSST)*, 2017, pp. 1–16.
- [13] W.-G. Liu, L.-F. Zeng, D. Feng, and K. B. Kent, "ROCO: Using a Solid State Drive Cache to Improve the Performance of a Host-Aware Shingled Magnetic Recording Drive," *Springer Journal of Computer Science and Technology (JCST)*, vol. 34, no. 1, pp. 61–76, 2019.
- [14] C. Ma, Z. Zhou, Y. Wang, Y. Wang, and R. Mao, "MAID-Q: Minimizing Tail Latency in Embedded Flash with SMR Disk via Q-Learning Model," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 11, pp. 3709–3720, 2022.
- [15] Z. Chen, G. Wang, Z. Shi, Y. Guan, and T. Wang, "Region-Based Flash Caching with Joint Latency and Lifetime Optimization in Hybrid SMR Storage Systems," in *Design, Automation, and Test in Europe (DATE)*, 2023, pp. 1–6.
- [16] W. Xiao, H. Dong, L. Ma, Z. Liu, and Q. Zhang, "HS-BAS: A Hybrid Storage System Based on Band Awareness of Shingled Write Disk," in *International Conference on Computer Design (ICCD)*, 2016, pp. 64–71.
- [17] C.-F. Wu, M. Kuo, M.-C. Yang, and Y.-H. Chang, "Performance Enhancement of SMR-Based Deduplication Systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 9, pp. 2835–2848, 2022.
- [18] Y.-P. Liang, T.-Y. Chen, C.-H. Chi, H.-W. Wei, and W.-K. Shih, "Enabling a B+-Tree-Based Data Management Scheme for Key-Value Store over SMR-Based SSHD," in *Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [19] S. Zhou, E. Xu, H. Wu, Y. Du, J. Cui, W. Fu, C. Liu, Y. Wang, W. Wang, S. Sun, X. Wang, B. Feng, B. Zhu, X. Tong, W. Kong, L. Liu, Z. Wu, J. Wu, Q. Luo, and J. Wu, "SMRSTORE: A Storage Engine for Cloud Object Storage on HM-SMR Drives," in *Conference on File and Storage Technologies (FAST)*, 2023, pp. 395–408.
- [20] S.-H. Chen, M.-C. Yang, Y.-H. Chang, and C.-F. Wu, "Enabling File-Oriented Fast Secure Deletion on Shingled Magnetic Recording Drives," in *Design Automation Conference (DAC)*, 2019, pp. 1–6.
- [21] Y. Pan, Z. Jia, Z. Shen, B. Li, W. Chang, and Z. Shao, "Reinforcement Learning-Assisted Cache Cleaning to Mitigate Long-Tail Latency in DM-SMR," in *Design Automation Conference (DAC)*, 2021, pp. 103–108.
- [22] L. Shi, J. Li, C. Jason Xue, and X. Zhou, "Hybrid Nonvolatile Disk Cache for Energy-Efficient and High-Performance Systems," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 18, no. 1, pp. 1–23, 2013.
- [23] J. Wan, N. Zhao, Y. Zhu, J. Wang, Y. Mao, P. Chen, and C. Xie, "High Performance and High Capacity Hybrid Shingled-Recording Disk System," in *International Conference on Cluster Computing (CLUSTER)*, 2012, pp. 173–181.
- [24] D. Luo, J. Wan, Y. Zhu, N. Zhao, F. Li, and C. Xie, "Design and Implementation of a Hybrid Shingled Write Disk System," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 27, no. 4, pp. 1017–1029, 2016.
- [25] X. Xie, T. Yang, Q. Li, D. Wei, and L. Xiao, "Duchy: Achieving Both SSD Durability and Controllable SMR Cleaning Overhead in Hybrid Storage Systems," in *International Conference on Parallel Processing (ICPP)*, 2018, pp. 1–9.
- [26] J. Bhimani, Z. Yang, J. Yang, A. Maruf, N. Mi, R. Pandurangan, C. Choi, and V. Balakrishnan, "Automatic Stream Identification to Improve Flash Endurance in Data Centers," *ACM Transactions on Storage (TOS)*, vol. 18, no. 2, pp. 1–29, 2022.
- [27] B. Li, C. Deng, J. Yang, D. Lilja, B. Yuan, and D. Du, "HAML-SSD: A Hardware Accelerated Hotness-Aware Machine Learning based SSD Management," in *International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.
- [28] C. Gao, M. Ye, Q. Li, C. J. Xue, Y. Zhang, L. Shi, and J. Yang, "Constructing Large, Durable and Fast SSD System via Reprogramming 3D TLC Flash Memory," in *International Symposium on Microarchitecture (MICRO)*, 2019, pp. 493–505.
- [29] Dharamjeet, Y.-S. Chen, T.-Y. Chen, Y.-H. Kuan, and Y.-H. Chang, "LLSM: A Lifetime-Aware Wear-Leveling for LSM-Tree on NAND Flash Memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 11, pp. 3946–3956, 2022.
- [30] S. Lee and J. Kim, "Improving Performance and Capacity of Flash Storage Devices by Exploiting Heterogeneity of MLC Flash Memory," *IEEE Transactions on Computers (TC)*, vol. 63, no. 10, pp. 2445–2458, 2014.
- [31] Samsung-Corporation, "K9G4G08U0F 2GB MLC NAND Flash Datasheet." [Online]. Available: <https://datasheetspdf.com/pdf/776555/Samsung/K9GAG08U0F/6>
- [32] "MSR Cambridge Block I/O Traces," 2024. [Online]. Available: <http://iotta.cs.hmc.edu/traces/388>